Task and Project Management Tool

Four-member implementation checklist - Java Spring Boot backend + separate HTML/CSS/JavaScript frontend

Project outcome

Deliver a backend application that lets managers organise projects and coordinate team work. Team members can view their permitted work and update the status of tasks assigned to them. The frontend is a separate, simple responsive layer that calls the REST API.

Phase	Completion target
1. Planning	Agree stack, database model, endpoint names, JSON fields, enums, date format, status transitions, roles, branches, and CORS/API base URL.
2. Skeleton	Create Spring Boot project and database connection; establish common errors; create frontend pages, CSS, and API helper.
3. Build	Implement project, task, user/membership, permission, test, and frontend modules in parallel.
4. Integrate	Verify create project -> create/assign task -> filter -> update status -> overdue and summary flows. Resolve CORS and contract mismatches.
5. Submit	Run tests; prepare ER diagram/schema, Postman cases, README, screenshots/demo, GitHub source, and viva demonstration.

Member 1 - Project and database

[ ] Design ER diagram/schema: User, Project, ProjectMember, Task; define keys, relationships, indexes.

[ ] Build Project entity, repository, service, controller; create, list, detail, update, archive endpoints.

[ ] Use an archived flag; add project summary with total, TODO, in-progress, done, and overdue counts.

[ ] Validate required project fields and test create, view, update, archive, and unknown IDs.

Member 2 - Task management and business rules

[ ] Build Task entity, repository, service, controller; enforce relationships to Project and User.

[ ] Create/list/update tasks; support filters: project, status, priority, assignee, due date.

[ ] Reject missing required fields, invalid project IDs, and due dates before start dates.

[ ] Enforce transitions: TODO -> IN_PROGRESS -> DONE; decide whether IN_PROGRESS -> TODO is allowed. Detect overdue tasks when due date is past and status is not DONE.

Member 3 - Users, permissions, and API testing

[ ] Build User and ProjectMember functions; prevent duplicate membership.

[ ] Managers/admins create, update, archive projects and manage assignment; members only see permitted projects and update their assigned task status.

[ ] Add service-layer permission checks. Add JWT/role-based authentication only after core APIs are stable.

[ ] Create Postman positive/negative cases and integration tests for fields, dates, IDs, filters, permissions, and status transitions.

Member 4 - Frontend and end-to-end integration

[ ] Create independent pages: dashboard, projects, project details, tasks; add shared CSS and JavaScript.

[ ] Build navigation, project list/forms, task table/form, status controls, filters, messages, and responsive layout.

[ ] Use fetch() to call the API. Send ISO dates and agreed enum values; backend validation remains authoritative.

[ ] Run complete end-to-end workflow and capture screenshots/demo evidence.

Shared API and validation checklist

Endpoint	Check
POST /api/projects	Manager-only creation; required fields and error response.
GET /api/projects and /api/projects/{id}	Visibility rules; details; archived project semantics.
PUT /api/projects/{id} and PATCH /api/projects/{id}/archive	Manager permission; valid ID; archive flag rather than hard delete.
POST /api/projects/{id}/tasks	Valid project; required task fields; assignee/membership decision.
GET /api/tasks?projectId&status;&priority;&assigneeId;&dueDate;	Every filter works alone and together; sorting/pagination optional.
PATCH /api/tasks/{id}/status	Assigned-user/manager permission and only approved state changes.
GET /api/projects/{id}/summary	Accurate total, TODO, in-progress, done, overdue counts.

Final delivery checklist

[ ] Working Spring Boot backend and integrated, separate HTML/CSS/JavaScript frontend.

[ ] GitHub repository with regular commits and clear ownership/branches.

[ ] ER diagram/database schema; API contract and assumptions.

[ ] Postman collection with successful and failing requests.

[ ] Unit and integration tests for core business rules.

[ ] README with setup steps, database configuration, API base URL, and test/demo instructions.

[ ] Demo flow and screenshots; optional extensions only after the core is complete.